Creating a home ‘Cloud’ service
- Lets me host my project with privacy first
- No monthly recurring costs

Process
- Used an ASUS laptop for the first machine
- Have three spare machines to use as a cluster
- Proxmox Virtual Environment (VE)
- Use Tailscale on my mac to connect to the three machine cluster

Personal docs[do not upload]:
Node 1
- ssh ubuntu@100.109.169.24

Node 2
- ssh ubuntu@100.71.245.120

Node 3
- ssh ubuntu@100.94.9.70

```
server_join {
  retry_join = ["192.168.254.49", "192.168.254.21", "192.168.254.73"]
}
```

Part 1:

---

1. The Core Architecture

You are replacing Windows with a professional-grade server operating system to avoid forced reboots and maximize hardware power.

- **The OS: Proxmox VE** (A Type-1 Hypervisor). It runs "bare metal" on the PCs.
- **The Network: Tailscale**. This acts as a magic, secure tunnel connecting your Mac to the remote machines from anywhere in the world, without touching router port forwarding.
- **The Workloads:** You will run your projects (like OpenClaw) inside lightweight **Linux Containers (LXC)** or **Virtual Machines (VMs)** on Proxmox.

2. Remote Location "Golden Rules"

Since these machines will live in a different location from you, you must follow these rules:

- **The "Same Roof" Rule:** All three PCs *must* be physically plugged into the same local network (router/switch) at the remote location so they can talk to each other without internet lag.

- **Power Outage Proofing:** You must go into each PC's BIOS and set **"Restore on AC Power Loss"** to **"Power On"** so they boot themselves back up if the remote location loses electricity.

3. The Installation (The flash drive route)

You are using your portable flash drive as the temporary "delivery truck" to install Proxmox onto the PC's internal drive.

> **CRITICAL WIPE WARNING:** > 1. Flashing the installer will **completely wipe everything** currently on your portable flash drive.
>
> 2. Installing Proxmox will **completely wipe Windows** and all files on the PC's internal drive.

The 4-Step Process:

1. Use your Mac and a program like **BalenaEtcher** to flash the Proxmox ISO onto your portable SSD.
2. Plug the SSD into the Windows PC and boot from it (via BIOS settings).
3. Follow the wizard to install Proxmox onto the PC's **internal drive**.
4. Unplug the portable SSD, let the PC reboot, and you are done with the SSD (you can reformat it for your Mac later).

4. The Rollout Strategy

Do not set this up for the first time at the remote location.

- **Phase 1:** Wipe *one* PC at home next to your Mac. Install Proxmox, set up Tailscale, and make sure you can log into the dashboard via your Mac's browser.
- **Phase 2:** Move that one working PC to the remote location and verify it comes back online.
- **Phase 3:** Wipe the other two PCs, move them, and join them to the first one to create your "Supermachine" cluster

What's currently done:

Having that dashboard load via the `100.x.x.x` IP address proves your secure remote architecture is fully functional.

Here is a clean, chronological summary of exactly what you accomplished for your documentation.

Phase 1: Node 1 Setup Documentation

1. Installation Media Prep (Mac)

- Downloaded the Proxmox VE ISO and BalenaEtcher.
- Flashed the OS image onto a 8GB USB drive, successfully ignoring macOS "unreadable disk" format warnings.

2. Hardware & BIOS Configuration (Laptop / Node 1)

- Connected the laptop to power and hardwired it to a Wi-Fi relay to satisfy the Proxmox wired-network requirement.
- Accessed the BIOS to disable **Secure Boot** and set the USB drive as the primary boot priority.
- Committed to keeping the laptop lid open to bypass sleep-state issues.

3. Proxmox OS Installation

- Booted into the Proxmox Graphical Installer via the USB drive.
- Targeted the laptop's internal drive for installation (wiping the existing Windows OS).
- Configured localization, set the master `root` password, and allowed the system to auto-assign a local IP address via the Wi-Fi relay.
- Ejected the USB drive and rebooted into the headless terminal.

4. Initial Access & Repository Configuration (Mac)

- Accessed the Proxmox web GUI using the local IP (`https://[LOCAL_IP]:8006`), bypassing the browser's self-signed SSL warning.
- Resolved `401 Unauthorized` update errors by navigating to **Datacenter > [Node Name] > Updates > Repositories**.
- Disabled the default `enterprise.proxmox.com` repositories and added the free **No-Subscription** repository.
- Opened the Proxmox Shell and ran `apt update` to sync the new software sources.

5. Tailscale Network Tunnel Setup

- Executed the official Tailscale install script in the Proxmox Shell: `curl -fsSL https://tailscale.com/install.sh | sh`.
- Ran `tailscale up` to start the service and generate a unique authentication link.
- Authenticated the Proxmox node via the Mac's browser to officially join it to the private Tailscale network.
- Retrieved the node's permanent Tailscale IP (`100.x.x.x`) from the Mac's Tailscale app.

6. Remote Connectivity Verification

- Successfully loaded the Proxmox dashboard from the Mac using the secure Tailscale IP (`https://100.x.x.x:8006`), verifying that remote management works independently of the local network.

Phase 2: GPU VM Deployment (Node 1)

7. Virtual Machine Creation (Ubuntu GPU Node)

Created a dedicated Ubuntu Server VM within Proxmox to host GPU workloads.

VM configuration:

Setting	Value
VM Name	gpu-node-1
VM ID	100
OS	Ubuntu 22.04 Server
BIOS	OVMF (UEFI)
Machine Type	q35
CPU Type	host
CPU Allocation	12 cores
Memory	12 GB

Disk	100 GB (local-lvm)
Network	VirtIO via vmbr0

The Ubuntu Server ISO (`ubuntu-22.04.5-live-server-amd64.iso`) was uploaded to Proxmox storage and attached to the VM for installation.

---

8. PCI Passthrough Configuration (GPU)

To enable GPU access inside the VM, PCI passthrough was configured on the Proxmox host.

Enable IOMMU

Edited GRUB configuration:

/etc/default/grub

Changed:

GRUB_CMDLINE_LINUX_DEFAULT="quiet"

To:

GRUB_CMDLINE_LINUX_DEFAULT="quiet intel_iommu=on iommu=pt"

Then applied changes:

update-grub

reboot

Verification:

dmesg | grep -e DMAR -e IOMMU

Confirmed:

DMAR: IOMMU enabled

Bind GPU to VFIO

Identified GPU PCI IDs:

lspci -nn | grep -i nvidia

Result:

10de:25a2 (RTX 3050)

10de:2291 (GPU audio)

Created VFIO configuration:

/etc/modprobe.d/vfio.conf

options vfio-pci ids=10de:25a2,10de:2291

Enabled VFIO modules:

/etc/modules

Added:

vfio

vfio_iommu_type1

vfio_pci

vfio_virqfd

Rebuilt initramfs:

update-initramfs -u -k all

reboot

Verification:

lspci -nnk -d 10de:

Confirmed both devices were using:

Kernel driver in use: vfio-pci

---

9. GPU Attachment to the VM

The GPU was attached to the VM using manual configuration.

Edited:

/etc/pve/qemu-server/100.conf

Added:

hostpci0: 0000:01:00.0,pcie=1

hostpci1: 0000:01:00.1,pcie=1

This exposed the RTX 3050 and its audio controller to the VM.

---

10. Ubuntu Installation

Booted the VM and installed Ubuntu Server.

Key selections:

Installer Step	Choice
Network	DHCP
Storage	Use entire disk with LVM
SSH	Install OpenSSH server
Featured snaps	None

The installation completed successfully and the system rebooted into the new OS.

---

11. GPU Visibility Verification

Logged into the VM and verified the GPU appeared at the PCI level.

Command:

lspci | grep -i nvidia

Output confirmed GPU presence:

NVIDIA GA107M [GeForce RTX 3050 Mobile]

---

12. NVIDIA Driver Installation

Installed the recommended driver automatically:

sudo ubuntu-drivers autoinstall

sudo reboot

After reboot, verified compute functionality:

nvidia-smi

The RTX 3050 was successfully detected with CUDA support.

Pain Points Encountered and Resolutions

GPU Not Appearing in Proxmox PCI Device List

Issue

The GPU did not appear in the Proxmox "Add PCI Device" dropdown.

Cause

The GPU was still bound to the host kernel driver instead of the VFIO passthrough driver.

Resolution

Bound the GPU to `vfio-pci` using:

/etc/modprobe.d/vfio.conf

and rebuilding the initramfs.

Proxmox Console Error (Failed to run vncproxy)

Issue

Opening the VM console returned:

Error: Failed to run vncproxy

Cause

The `x-vga=1` option caused the GPU to fully replace the VM display device, breaking Proxmox's VNC console.

Resolution

Removed the `x-vga=1` parameter from the VM configuration.

Empty PCI Device Dropdown

Issue

Even after enabling IOMMU, the GPU did not appear in the Proxmox GUI.

Cause

Proxmox UI occasionally hides devices already bound to VFIO.

Resolution

Manually added the GPU to the VM configuration file (`100.conf`).

Ubuntu Installer Hanging at `subiquity/Late/run`

Issue

Installer appeared stuck during the final stage.

Cause

The installer was completing background configuration tasks.

Resolution

Rebooted safely using the installer reboot option; installation had already completed successfully.

Current Node Status

Node 1 is now fully operational with:

- Proxmox hypervisor
- Remote management via Tailscale
- Ubuntu GPU VM
- RTX 3050 passed through
- NVIDIA drivers installed
- CUDA compute available

This node can now host GPU workloads such as:

- LLM inference
- Stable Diffusion
- PyTorch / CUDA training
- Docker-based AI services.

Node Setup Checklist (Reusable for Nodes 2 & 3)

Host (Proxmox)

1. Install **Proxmox VE** on the machine (wipe existing OS).
2. Access dashboard via local IP:

```
https://[LOCAL_IP]:8006
```

3. Disable enterprise repos and enable **no-subscription repo**.
4. Install **Tailscale**:

```
curl -fsSL https://tailscale.com/install.sh | sh
tailscale up
```

5. Verify remote access via Tailscale:

```
https://100.x.x.x:8006
```

Enable GPU Passthrough

6. Enable IOMMU:

```
nano /etc/default/grub
GRUB_CMDLINE_LINUX_DEFAULT="quiet intel_iommu=on iommu=pt"
update-grub
reboot
```

7. Identify GPU PCI IDs:

```
lspci -nn | grep -i nvidia
```

8. Bind GPU to VFIO:

```
nano /etc/modprobe.d/vfio.conf
options vfio-pci ids=<GPU_ID>,<AUDIO_ID>
```

9. Enable VFIO modules:

```
nano /etc/modules
vfio
vfio_iommu_type1
vfio_pci
vfio_virqfd
```

10. Apply and reboot:

```
update-initramfs -u -k all
reboot
```

Create GPU VM

11. Upload **Ubuntu Server ISO** to Proxmox storage.
12. Create VM:

Setting	Value
BIOS	OVMF (UEFI)
Machine	q35
CPU Type	host
CPU	~70–80% host cores

RAM	~70–80% host RAM
Disk	100GB+
Network	VirtIO / vmbr0

Attach GPU

13. Edit VM config:

```
/etc/pve/qemu-server/<VMID>.conf
```

Add:

```
hostpci0: 0000:01:00.0,pcie=1
hostpci1: 0000:01:00.1,pcie=1
```

Install Ubuntu

14. Boot VM and install Ubuntu Server.
15. Enable **OpenSSH server** during install.

GPU Driver

16. Verify GPU presence:

```
lspci | grep -i nvidia
```

17. Install driver:

sudo ubuntu-drivers autoinstall

sudo reboot

18. Verify compute:

nvidia-smi

If successful, the node is ready for GPU workloads.

Phase 3: Container Runtime & GPU Workload Verification

13. Docker Installation

Installed Docker inside the Ubuntu VM to run containerized workloads.

sudo apt install -y docker.io

Verified installation:

docker run hello-world

The container executed successfully, confirming Docker was operational.

---

14. Docker Permission Configuration

Initial Docker test produced a permission error:

connect: permission denied

Cause

Docker socket access (`/var/run/docker.sock`) is restricted to the `docker` group.

Resolution

Added the current user to the Docker group:

```
sudo usermod -aG docker $USER
```

Logged out and logged back into the VM to apply group permissions.

Docker commands were then executable without `sudo`.

---

15. NVIDIA Container Toolkit Installation

Added NVIDIA container runtime repository.

```
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | \
sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list | \
sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg] https://#g' | \
sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
```

Updated package lists and installed toolkit:

```
sudo apt update
sudo apt install -y nvidia-container-toolkit
```

Configured Docker runtime:

```
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

---

16. GPU Container Validation

Executed a CUDA container to verify GPU access inside Docker.

```
docker run --rm --gpus all nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi
```

Docker automatically pulled the CUDA base image and executed `nvidia-smi`.

Output confirmed:

- RTX 3050 detected
- NVIDIA driver loaded
- CUDA runtime accessible from container

This validated the complete GPU stack:

GPU Hardware

↓

VFIO passthrough

↓

Ubuntu VM

↓

NVIDIA Driver

↓

CUDA

↓

Docker

↓

NVIDIA Container Runtime

↓

GPU-enabled containers

Pain Points Encountered

Docker Permission Denied

Issue

connect: permission denied

Cause

User not in the `docker` group.

Resolution

Added user to docker group and re-authenticated session.

NVIDIA Container Toolkit Package Not Found

Issue

E: Unable to locate package nvidia-container-toolkit

Cause

NVIDIA package repository was not configured.

Resolution

Added NVIDIA repository and installed toolkit successfully.

CUDA Container Image Not Found

Issue

Unable to find image locally

Cause

Docker had not yet downloaded the CUDA image.

Resolution

Docker automatically pulled the image from Docker Hub.

Converted the first one to a template for the others

Attached the other two to the network switch

Phase 4: Node-2 Deployment and Cluster Formation

17. Node-2 Proxmox Installation

Installed Proxmox VE on the second machine (desktop PC).

Steps performed:

1. Reflashed the Proxmox ISO onto the same USB installer used for Node-1.
2. Booted Node-2 from the USB device via the BIOS boot menu.
3. Launched the **Proxmox Graphical Installer**.

Key installation choices:

Setting	Value
Hostname	pve-node2
Disk	Internal system disk (Windows wiped)
Network	DHCP
Management Interface	nic0

Installation completed successfully and the system rebooted into the Proxmox host.

---

18. Initial Node-2 Access

Accessed the Node-2 Proxmox dashboard via the browser:

https://<node2_local_ip>:8006

Logged in using:

user: root

password: <same root password used for node1>

Verified connectivity and opened the Proxmox shell.

---

19. Repository Configuration (Node-2)

Disabled enterprise repositories and enabled the free community repository.

Path:

Datacenter → pve-node2 → Updates → Repositories

Actions:

1. Disabled `enterprise.proxmox.com`
2. Added **No-Subscription repository**

Updated package index:

apt update

---

20. Tailscale Installation (Node-2)

Installed Tailscale to enable remote management identical to Node-1.

curl -fsSL https://tailscale.com/install.sh | sh

tailscale up

Authenticated through the browser.

Verified connection:

tailscale status

Node-2 received a new **100.x.x.x** address and appeared in the Tailscale network.

---

21. Hostname Conflict Resolution

While attempting to join the cluster, an error occurred:

can't add existing node 'pve'

Cause

Both machines shared the hostname `pve`.

Resolution

Renamed Node-2:

hostnamectl set-hostname pve-node2

Updated the hosts file:

/etc/hosts

Then rebooted the system.

After reboot the node appeared correctly as:

pve-node2

---

22. Cluster Creation (Node-1)

Created a Proxmox cluster from the first node.

Location:

Datacenter → Cluster

Cluster name:

home-cluster

Cluster initialization completed successfully.

---

23. Node-2 Cluster Join

Obtained cluster join information from Node-1:

Datacenter → Cluster → Join Information

Then on Node-2:

Datacenter → Cluster → Join Cluster

Pasted the join information and entered the root password for Node-1.

Node-2 successfully joined the cluster.

Cluster view now displayed:

Datacenter

├ pve-node1

└ pve-node2

---

24. Cluster Certificate Sync Issue

After joining, the following error appeared:

/etc/pve/nodes/pve-node2/pve-ssl.pem does not exist

Cause

Cluster filesystem had not generated SSL certificates yet.

Resolution

Regenerated cluster certificates:

pvecm updatecerts --force

systemctl restart pveproxy

Error cleared after refreshing the dashboard.

Phase 5: Node-2 GPU Deployment

25. GPU Detection

Verified Node-2 GPU from the Proxmox host:

lspci | grep -i nvidia

Detected hardware:

NVIDIA GeForce GTX 1060 3GB

---

26. GPU Passthrough Configuration

Enabled IOMMU in GRUB:

/etc/default/grub

Modified:

GRUB_CMDLINE_LINUX_DEFAULT="quiet intel_iommu=on iommu=pt"

Applied changes:

update-grub

reboot

---

27. VFIO Binding

Identified GPU PCI IDs:

lspci -nn | grep -i nvidia

Configured VFIO driver binding:

/etc/modprobe.d/vfio.conf

Example:

options vfio-pci ids=<GPU_ID>,<AUDIO_ID>

Enabled VFIO modules:

/etc/modules

vfio

vfio_iommu_type1

vfio_pci

vfio_virqfd

Rebuilt initramfs:

update-initramfs -u -k all

reboot

Verified:

lspci -nnk -d 10de:

Output confirmed:

Kernel driver in use: vfio-pci

---

28. Template Deployment to Node-2

Direct cloning to Node-2 failed because the template resided on **local storage**.

Cause

Proxmox cannot clone directly across nodes without shared storage.

Resolution

1. Cloned the template locally on Node-1:

VM 100 → Clone

Target Node: pve-node1

Mode: Full Clone

2. Migrated the new VM to Node-2.

Initial migration failed due to attached PCI devices:

Cannot migrate VM with local resources: hostpci0, hostpci1

Removed Node-1 GPU passthrough entries from the VM configuration.

Migration then completed successfully.

---

29. Node-2 GPU Attachment

Edited VM configuration on Node-2:

/etc/pve/qemu-server/102.conf

Added:

hostpci0: 0000:01:00.0,pcie=1

hostpci1: 0000:01:00.1,pcie=1

Started the VM.

Inside the VM verified GPU availability:

nvidia-smi

Output confirmed:

GeForce GTX 1060

---

30. Runtime Verification

Confirmed containerized GPU workload capability:

docker run --rm --gpus all nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi

The container detected the GTX 1060 successfully, confirming full CUDA runtime support.

Current Cluster Status

The environment now consists of:

Cluster: home-cluster

Node1 → RTX 3050 GPU VM

Node2 → GTX 1060 GPU VM

Both nodes:

- run identical VM environments cloned from the template
- support Docker GPU workloads
- are managed from a single Proxmox cluster dashboard.

Phase 6: Node-3 Deployment

31. Proxmox Installation (Node-3)

Attempted to install Proxmox VE on the third machine.

During the installer boot process the system **hung before launching the graphical installer**.

Resolution

Edited the boot parameters from the Proxmox boot screen.

Steps:

1. At **"Welcome to Proxmox"** screen pressed e
2. Modified the kernel boot line:

linux /boot/linux26 ro quiet splash=silent

to include:

nomodeset

Final line:

linux /boot/linux26 ro quiet splash=silent nomodeset

This disabled GPU framebuffer initialization and allowed the installer to launch.

The installation then proceeded normally.

---

32. Hostname Configuration Issue

During installation the hostname was set to:

pve-node3

However after installation the node appeared as:

pve

in the Proxmox environment.

Cause

The hostname was not correctly propagated to the system configuration files during installation.

Resolution

The hostname will be corrected manually before joining the cluster.

Commands to run on Node-3:

hostnamectl set-hostname pve-node3

Update hosts file:

nano /etc/hosts

Change entry:

192.168.xxx.xxx pve

to:

192.168.xxx.xxx pve-node3

Then reboot.

---

33. Tailscale Installation

Installed Tailscale on Node-3 to enable remote management.

```
curl -fsSL https://tailscale.com/install.sh | sh
tailscale up
```

---

34. Added pve-node3 to cluster

Added pve-node3 to cluster using the join information, appears correctly on tailscale as well

Phase 7: Node-3 Integration & GPU Finalization

35. GPU Hardware Initialization (Host)

Before attaching the GPU to the VM, the host environment on pve-node3 was prepared for hardware passthrough.

- **IOMMU Activation:** Edited /etc/default/grub to include intel_iommu=on iommu=pt.
- **Kernel Refresh:** Executed update-grub and rebooted the host.
- **VFIO Binding:** Identified the GPU PCI IDs using lspci -nn | grep -i nvidia and bound them to the vfio-pci driver via /etc/modprobe.d/vfio.conf.
- **Module Loading:** Added vfio, vfio_iommu_type1, vfio_pci, and vfio_virqfd to /etc/modules.

36. Template Deployment & Cloning

To maintain environment parity across the cluster, a pre-configured Ubuntu GPU template was deployed to Node 3.

- **Target Node Migration:** Cloned the master template from Node 1 and migrated the disk resources to Node 3's local storage.
- **Hardware Mapping:** Manually edited the VM configuration file (/etc/pve/qemu-server/[VM_ID].conf) to map the specific PCI addresses of Node 3's GPU (0000:01:00.0 and 0000:01:00.1).

37. Guest OS Identity Correction

Because the VM was cloned from a Node 1 template, the internal identity required updating to avoid network confusion.

- **Hostname Update:** Executed sudo hostnamectl set-hostname gpu-node-3.
- **Network Resolution:** Updated /etc/hosts to reflect the change from gpu-node-1 to gpu-node-3.
- **Proxmox Sync:** Updated the VM name in the Proxmox "Options" tab to ensure the GUI sidebar matched the terminal prompt.

38. Cluster-Wide GPU Verification

Verified the complete stack on the final node.

- **NVIDIA-SMI:** Confirmed the driver recognized the GPU and CUDA was available.
- **Container Test:** Ran the validation container:
  docker run --rm --gpus all nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi
- **Result:** Successful execution across all three nodes.

Current "Supermachine" Cluster Architecture

Node Name	Hardware Type	Primary GPU	Role in AI Projects
pve-node1	Laptop	RTX 3050 Mobile	UI, FastAPI Orchestrator, Quorum Vote
pve-node2	Desktop	GTX 1060 3GB	Model Inference, Dev Environment
pve-node3	Desktop	RTX 3080	Heavy Training Engine, Batch Processing

Operational Status

- **Remote Management:** 100% accessible via **Tailscale** (100.x.x.x addresses) from your MacBook Air.
- **Orchestration:** All three nodes are joined to home-cluster, allowing for single-pane-of-glass management.
- **Environment:** All nodes run identical Ubuntu 22.04 VMs with Docker and NVIDIA Container Toolkit pre-installed

---

Part 2: Auto selection of workloads

PREREQ:
Enabling SSH Access to Proxmox Ubuntu VMs Using Tailscale

Overview

This document explains how to enable secure SSH access to Ubuntu virtual machines using Tailscale.
Each VM must authenticate with Tailscale individually to obtain a Tailscale network IP (100.x.x.x).

Once authenticated, the VM can be accessed remotely via SSH.

Prerequisites (for PREREQ)

- Ubuntu VM running
- Tailscale installed on the VM
- Access to the VM console through Proxmox
- Tailscale account (Google login in this case)
- Nomad CLI

Step 1 — Verify Tailscale Status

Open the VM console in Proxmox and run:

tailscale status

If Tailscale is not connected, you will see a message indicating the node is logged out or inactive.

Step 2 — Authenticate the VM with Tailscale

Run:

sudo tailscale up

The command will output a login URL similar to:

https://login.tailscale.com/a/XXXXXXXX

Open this link in your browser and authenticate using the same account used for your Tailscale network (Google login).

After authentication, the VM will join your Tailnet.

Step 3 — Confirm Tailscale IP Address

Run:

tailscale ip

You should receive an IP address in the following format:

100.x.x.x

This is the secure internal Tailscale IP assigned to the VM.

Step 4 — Enable SSH Password Authentication

Edit the SSH configuration:

sudo nano /etc/ssh/sshd_config

Find the following line:

PasswordAuthentication

Ensure it is set to:

PasswordAuthentication yes

Save the file.

Restart the SSH service:

```
sudo systemctl restart ssh
```

Step 5 — Connect to the VM via SSH

From your local machine, run:

```
ssh ubuntu@100.x.x.x
```

Enter the Ubuntu user password when prompted.

Note: The terminal will not display characters while typing the password.

Step 6 — Enable Tailscale on Boot

To ensure Tailscale starts automatically after reboot:

```
sudo systemctl enable tailscaled
```

Step 7 — Repeat for Additional Nodes

Each VM must authenticate separately.

Repeat Steps 1–6 for:

Node 1
Node 2
Node 3

After completing this process, all nodes will appear in the Tailscale admin console and can securely communicate with each other.

Verification

You can verify connectivity by SSHing into each node using its Tailscale IP:

ssh ubuntu@100.x.x.x

If successful, you will receive the Ubuntu shell prompt.

Notes

• Each VM is treated as a separate device in Tailscale.
• Authentication is only required once per VM.
• Tailscale provides secure networking without exposing the VM to the public internet.

HashiCorp Nomad

Nomad is like a much simpler, more flexible version of Kubernetes. It is very popular for small home labs.

- **How it works:** It's a single binary you run on all nodes. It handles "Job Scheduling" and natively understands NVIDIA GPUs very well.
- **Difficulty: 6/10.** It's easier to set up than Kubernetes but more robust than a custom Python script.
- **The Reward:** Great balance of "set it and forget it" without the massive overhead of K8s

Step 1: Install Nomad on all 3 Nodes

You need to run these commands on the Proxmox Shell of **Node 1, Node 2, and Node 3**. Nomad is just a single binary, so the installation is identical for all.

Add the HashiCorp GPG key:
Bash
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg

Add the official repository:
Bash
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]

https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list

Install Nomad:
Bash
sudo apt update && sudo apt install nomad -y

Step 2: Configure Node 1 as the "Server+Client" (The Leader)

Your laptop will manage the schedule. It won't run the heavy AI jobs itself; it just tells the desktops what to do.

1. **Edit the config:** `sudo nano /etc/nomad.d/nomad.hcl`

Replace the contents with this:
Terraform
```
data_dir  = "/opt/nomad/data"

bind_addr = "0.0.0.0" # Listen on all interfaces



server {

  enabled          = true

  bootstrap_expect = 1 # We only need 1 leader for this small cluster

}

client {

  enabled = true

}

plugin "docker" {

  config {

    allow_privileged = true

    allow_caps = ["ALL"]

  }
```

```
}

advertise {

  # Use your Node 1 Tailscale IP here

  http = "100.x.x.x"

  rpc  = "100.x.x.x"

  serf = "100.x.x.x"

}
```

2.

Start it up:
Bash
```
sudo systemctl enable nomad

sudo systemctl start nomad
```

3.

Step 3: Configure Node 2 & 3 as "Clients" (The Muscle)

These machines will actually talk to the GPUs.

1. **Edit the config on Node 2 (and later Node 3):** `sudo nano /etc/nomad.d/nomad.hcl`

Use this Client config:
Terraform
```
data_dir = "/opt/nomad/data"

bind_addr = "0.0.0.0"



client {

  enabled = true

  # Point this to Node 1's Tailscale IP
```

```
  servers = ["100.NODE_1_IP:4647"]
}

plugin "docker" {
  config {
    allow_privileged = true
    allow_caps = ["ALL"]
  }
}

advertise {
  # Use the Tailscale IP of the CURRENT node (Node 2 or 3)
  http = "100.x.x.x"
  rpc  = "100.x.x.x"
  serf = "100.x.x.x"
}
```

2.
3. **Start it up:** `sudo systemctl enable nomad && sudo systemctl start nomad`

Step 4: The Moment of Truth

Go back to your **Mac terminal** and type: `nomad node status -address=http://100.NODE_1_IP:4646`

If you see all three nodes listed with a status of **"ready"**, your auto-scheduler is live.

Note: For mac make address permanent via:

nano ~/.zshrc

Add this line:

export NOMAD_ADDR="http://100.NODE1_IP:4646"

Reload:

source ~/.zshrc

Step 5: Install the NVIDIA Device Plugin

Nomad does not detect GPUs out of the box. You need to install a separate plugin binary on every node that tells Nomad's scheduler about the available GPUs. Without this, Nomad has no idea GPUs exist even if the NVIDIA drivers and Docker runtime are working perfectly.

Run the following on **all 3 nodes**:

Create the plugin directory:

bash

sudo mkdir -p /opt/nomad/plugins

Download and install the plugin binary:

bash

wget https://releases.hashicorp.com/nomad-device-nvidia/1.1.0/nomad-device-nvidia_1.1.0_linux_amd64.zip

unzip nomad-device-nvidia_1.1.0_linux_amd64.zip

sudo mv nomad-device-nvidia /opt/nomad/plugins/

sudo chmod +x /opt/nomad/plugins/nomad-device-nvidia

rm nomad-device-nvidia_1.1.0_linux_amd64.zip

Now update `/etc/nomad.d/nomad.hcl` on each node to tell Nomad where the plugin lives and to enable it. Add these two blocks to the existing config:

hcl

```
plugin_dir = "/opt/nomad/plugins"


plugin "nomad-device-nvidia" {
  config {
    enabled            = true
    fingerprint_period = "1m"
  }
}
```

Restart Nomad on each node after saving:

```bash
sudo systemctl restart nomad
```

To confirm the plugin loaded correctly, check the logs on any node:

```bash
sudo journalctl -u nomad | grep -i nvidia
```

You should see a line like:

```
agent: detected plugin: name=nvidia-gpu type=device plugin_version=1.1.0
```

Verify GPU Registration from your Mac:

Run this for each node ID:

```bash
nomad node status -verbose <node-id>
```

You are looking for a **Device Group Capacities** section that was not there before:

```

Device Group Capacities

Device Group                        Count  Healthy  Unhealthy

nvidia/gpu/NVIDIA GeForce RTX 3080    1       1        0
```

All three nodes should report their respective GPUs. At this point Nomad's scheduler is fully GPU-aware and will only place GPU jobs on nodes that have an available device.

Step 6: Run Your First GPU Job

Now that the scheduler knows about your GPUs, you need to request them correctly in your job file. The key difference from a normal job is the `device` stanza inside `resources`. This is what tells Nomad to reserve a GPU for the task and pass it through to the Docker container automatically.

Create a file called `gpu-test.nomad` on your Mac:

```hcl
job "gpu-test" {
  datacenters = ["dc1"]
  type        = "batch"


  group "gpu-group" {
    task "cuda-check" {
      driver = "docker"


      config {
        image   = "nvidia/cuda:12.4.1-base-ubuntu22.04"
```

```
      command = "nvidia-smi"
    }

    resources {
      cpu    = 500
      memory = 512

      device "nvidia/gpu" {
        count = 1
      }
    }
  }
 }
}
```

Run it:

bash

```
nomad job run gpu-test.nomad
```

Grab the allocation ID from the output and check the result:

bash

```
nomad alloc logs <alloc-id>
```

A successful run will print the full `nvidia-smi` table showing the GPU name, driver version, CUDA version, and memory. The task will show as `dead` with `Exit Code: 0` — this is expected and correct for batch jobs in Nomad. Dead means finished, not failed.

Important distinctions to remember:

- `type = "batch"` is for one-shot tasks like this test. The task goes `dead` when it completes successfully.

- `type = "service"` is what you will use for long-running workloads like inference servers. Those stay `running`.
- Never use `device_requests` inside the Docker `config` block — that is a raw Docker API field that Nomad ignores entirely. Always use the `device` stanza inside `resources`.

Couple of things to finish the setup:

VMs are not set to autostart in Proxmox

Your doc sets BIOS to restore power, but that only boots the Proxmox host. The Ubuntu VMs inside won't start automatically unless you configure this in Proxmox for each VM:

```
bash# On each Proxmox host, for each VM (replace 100 with your VM ID)

qm set 100 --onboot 1



# Verify

qm config 100 | grep onboot

# Should return: onboot: 1
```

Without this, a power outage brings Proxmox back up but all your VMs sit there powered off, and nothing runs.

Single Nomad server is a silent cluster killer

Node1 is your only Nomad server. If it goes offline — power, crash, VM issue — Nodes 2 and 3 go leaderless. Running jobs continue briefly but no new scheduling happens, and service jobs won't restart if they crash. You have three nodes so you can fix this properly:

```
hcl# On Node2 and Node3, add server block to nomad.hcl

server {

  enabled          = true

  bootstrap_expect = 3  # Change Node1's bootstrap_expect to 3 as well

}
```

Then restart Nomad on all three nodes. Now you can lose any one node and the cluster keeps working. This is the single most important change before running real workloads.

Part 3: README: How to use

https://github.com/kocierik/mcp-nomad

Restart policy on jobs

Current Nomad config has no restart or reschedule behavior defined. A crashed service job will stay dead. **Add this to every service job** you deploy:

```
group "your-group" {
  restart {
    attempts = 5
    interval = "5m"
    delay    = "15s"
    mode     = "delay"   # keep retrying, don't give up
  }

  reschedule {
    delay          = "30s"
    delay_function = "exponential"
    max_delay      = "10m"
    unlimited      = true   # keep trying to place on another node if current node dies
  }

  task "your-task" { ... }
}
```

Example Usage:

```
job "my-awesome-app" {

  datacenters = ["dc1"]

  type        = "service"


  group "web-nodes" {

    count = 3


    # --- ADD THESE HERE ---

    restart {

      attempts = 5

      interval = "5m"

      delay    = "15s"

      mode     = "delay"

    }


    reschedule {

      delay          = "30s"

      delay_function = "exponential"

      max_delay      = "10m"

      unlimited      = true

    }

    # ----------------------


    task "server" {
```

```
      driver = "docker"

      config {

        image = "nginx:latest"

      }

    }

  }

}
```

Persistent storage

Add this to every service job

Just like the restart/reschedule policies, this goes inside your Job file (the .nomad file for your specific app), specifically within the config block of your task.

2. Prepare the Nodes (Crucial Step) [example apollo]

Before you run a job with a volume, the folder must exist on the physical machine. Since Nomad could schedule your job on any of your three nodes, you should create the directory on all of them:

Run this on Node 1, Node 2, and Node 3:

bash

```
sudo mkdir -p /opt/nomad/volumes/apollo

# Give Nomad/Docker permission to write to it

sudo chmod -R 777 /opt/nomad/volumes/apollo
```

3. Add to your Job File

When you're ready to deploy your app (like Apollo-AI), your task block will look like this:

hcl

```
task "apollo-task" {
```

```
  driver = "docker"


  config {
    image = "100.NODE1_IP:5000/apollo-ai:latest"


    volumes = [
      # host_path : container_path
      "/opt/nomad/volumes/apollo:/app/data"
    ]
  }
}
```

4. The "Catch" with local volumes

Since you have a 3-node cluster now, there is one thing to keep in mind: Local volumes don't travel.

If your job runs on Node 2 and saves data to /opt/nomad/volumes/apollo, that data is physically on Node 2's hard drive.

If Node 2 crashes and Nomad moves the job to Node 3, the app will look in /opt/nomad/volumes/apollo on Node 3 and find... nothing (it's a fresh, empty folder).

How to handle this: Use constraints to force the job to always run on the same node where the data lives.

GTX 1060 has only 3GB VRAM

Add this to every service job

Worth noting explicitly: Node2's GTX 1060 3GB will fail silently for most AI inference workloads. Most models need at least 4-8GB. Use constraints in GPU jobs so they don't accidentally land there:

```
hcldevice "nvidia/gpu" {

  count = 1

  constraint {

    attribute = "${device.attr.memory}"

    operator  = ">="

    value     = "4 GiB"

  }

}
```

This ensures GPU jobs only land on Node1 (RTX 3050 Mobile) or Node3 (RTX 3080).

No health checks defined

Nomad can't tell if your service is actually working vs just running. **Add this to every service job**:

```
hclservice {

  name = "apollo-bot"



  check {

    type     = "script"

    command  = "/bin/sh"

    args     = ["-c", "pgrep -f your_process || exit 2"]

    interval = "30s"
```

```
    timeout  = "5s"

  }

}
```

For HTTP services use type = "http" with a /health endpoint instead.

Docker registry

A private Docker registry runs as a permanent Nomad service job on gpu-node-1. It is the bridge between writing code on your Mac and running it on the cluster — without it, Nomad can only pull public images. With it, you can build your own project images locally and deploy them to any node.

1. Create Persistent Storage for Registry Images

Images pushed to the registry must survive container restarts. Create a directory on Node 1 to store them:

bashsudo mkdir -p /opt/registry/data

sudo chown -R 1000:1000 /opt/registry/data

2. Deploy the Registry Job

Create registry.nomad:

```
hcljob "registry" {

  datacenters = ["dc1"]

  type        = "service"



  group "registry" {

    constraint {

      attribute = "${node.unique.id}"

      value     = "5bcd9b82-365b-aebd-538a-b54c5035967f"  # gpu-node-1

    }
```

```
network {
  port "registry" {
    static = 5000
    to     = 5000
  }
}

task "registry" {
  driver = "docker"

  config {
    image = "registry:2"
    ports = ["registry"]
    volumes = [
      "/opt/registry/data:/var/lib/registry"
    ]
  }

  resources {
    cpu    = 200
    memory = 256
  }
}
```

```
  }

}
```

bashnomad job run registry.nomad

3. Configure Docker to Trust the Registry (Mac + All Nodes)

Docker requires HTTPS by default. Since this registry runs over the Tailscale VPN without SSL, it must be explicitly trusted everywhere.

On your Mac: open Docker Desktop → Settings → Docker Engine and add:

```
json{

  "insecure-registries": ["100.NODE1_IP:5000"]

}
```

Click Apply & Restart.

On each Ubuntu node (Node 1, 2, and 3):

bashsudo nano /etc/docker/daemon.json

Add the same content, then restart Docker:

bashsudo systemctl restart docker

4. Verify It Works

```
bash# From your Mac — should return {}

curl http://100.NODE1_IP:5000/v2/


# Test push

docker pull alpine:latest

docker tag alpine:latest 100.NODE1_IP:5000/alpine:latest

docker push 100.NODE1_IP:5000/alpine:latest


# Confirm it's stored
```

```
curl http://100.NODE1_IP:5000/v2/_catalog

# Expected: {"repositories":["alpine"]}
```

5. How to Use It for Your Own Projects

```
bash# On your Mac, inside any project folder

docker build -t 100.NODE1_IP:5000/your-project:latest .

docker push 100.NODE1_IP:5000/your-project:latest
```

Then reference that image in your .nomad job file:

```
hclconfig {

  image = "100.NODE1_IP:5000/your-project:latest"

}
```

Nomad will pull it automatically on whichever node the job lands on.

Note on the network_interface fix: The registry initially bound to 127.0.0.1:5000 (loopback only) because Nomad didn't know which network interface to use. The fix was adding network_interface = "tailscale0" to the client block in /etc/nomad.d/nomad.hcl on all three nodes. This must be done on any new node added to the cluster.